Migrations and Alembic

- Due No Due Date
- Points 1
- Submitting a website url

 (https://github.com/learn-co-curriculum/python-p3-migrations-and-alembic)  (https://github.com/learn-co-curriculum/python-p3-migrations-and-alembic/issues/new)

Learning Goals

- Use an external library to simplify tasks from earlier ORM lessons.
- Manage database tables and schemas without ever writing SQL through Alembic.
- Use SQLAlchemy to create, read, update and delete records in a SQL database.

Key Vocab

- **Schema**: the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
- **Persist**: save a schema in a database.
- **Engine**: a Python object that translates SQL to Python and vice-versa.
- **Session**: a Python object that uses an engine to allow us to programmatically interact with a database.
- **Transaction**: a strategy for executing database statements such that the group succeeds or fails as a unit.
- **Migration**: the process of moving data from one or more databases to one or more target databases.

Introduction

You may have noticed in the curriculum so far that we haven't made many changes to the database schemas after we started adding data. If you've explored a bit more on your own, you probably ran into an error telling you that your new column doesn't exist, or even worse, that an old column cannot be accessed anymore.

```
# => TypeError: 'name' is an invalid keyword argument for Dog
```

This is a familiar problem to most professional programmers, managing **migrations**.

Alembic is a library for handling schema changes that uses SQLAlchemy to perform the migrations in a standardized way. Since SQLAlchemy only creates missing tables when we use the `Base.metadata.create_all()` method, it doesn't update the tables to match any changes we made to the columns or keys. Alembic provides us with classes and methods that will manage schema changes over the course of development. Because Alembic builds upon the functionality of SQLAlchemy, it can be used with a wide range of databases and web frameworks.

▶ *Which class do you need to import to run `Base.metadata.create_all()` ?*

Creating a Migration Environment

To create a migration environment, create a directory `app` and `cd` into that directory. Next, run `alembic init migrations` command to create a migration environment in the `migrations/` directory. This process creates our migration environment as well as an `alembic.ini` file with configuration options for the environment. If you run `tree` in our directory now, you should see this structure:

```
.
├── alembic.ini
└── migrations
    ├── README
    ├── env.py
    ├── script.py.mako
    └── versions
```

The `versions/` directory will hold our migration scripts. `env.py` defines and instantiates a SQLAlchemy engine, connects to that engine, starts a transaction, and calls the migration engine. `script.py.mako` is a template that is used when creating a migration- it defines the basic structure of a migration.

Now that we have created our environment, we need to configure it to work with our SQLAlchemy app.

Configuring a Migration Environment

`alembic.ini` and `env.py` contain important settings that need to be changed to work with our database and SQLAlchemy app specifically. `alembic.ini` contains a `sqlalchemy.url` setting on line 58 that points to the project database. Since we're starting to make changes to existing databases, we're going to use a `.db` file instead of working in memory:

```
# alembic.ini
sqlalchemy.url = sqlite:///migrations_test.db
```

Next, navigate into `models.py` to start designing our database with SQLAlchemy.

```
# models.py

#!/usr/bin/env python3
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base

engine = create_engine('sqlite:///migrations_test.db')

Base = declarative_base()
```

Next, we need to configure `env.py` to point to the metadata attribute of our new `declarative_base` object. Alembic will use this metadata to compare the structure of the database schema to the models as they are defined in SQLAlchemy. Now, let's update `migrations/env.py` :

```
# migrations/env.py
# search file for target_metadata (line 21)
from models import Base
target_metadata = Base.metadata
```

We're all set and ready to make our first migrations. Before we move onto the next section, run `tree` from the `app/` directory and make sure your directory structure matches the one below:

```
.
├── alembic.ini
└── migrations
    ├── README
    ├── env.py
    ├── script.py.mako
    └── versions
└── models.py
```

Generating our First Migration

Let's start off with creating a base, empty migration. Make sure you are in the `app/` directory and run the following command:

```
% alembic revision -m "Empty Init"
  Generating .../python-p3-migrations-and-alembic/app/migrations/versions/6b9cb35ba46e_empty_init.py ...  done
```

You should notice that a new file has popped up in the `migrations/versions/` directory. Here's what you should see inside:

```
"""Empty Init

Revision ID: 6b9cb35ba46e
Revises:
Create Date: 2022-08-04 13:21:26.936909

"""
from alembic import op
import sqlalchemy as sa
```

```
# revision identifiers, used by Alembic.
revision = '6b9cb35ba46e'
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
    pass


def downgrade() -> None:
    pass
```

This file starts off with the message that we included with our `alembic` command. It is important to treat these messages as you would commit messages in Git so that other developers know when certain tables, columns, keys, and so on were added to the database.

The `upgrade()` method includes the code that would be needed to perform changes to the database based on this migration. Similarly, the `downgrade()` method includes any code that would be needed to undo this migration and return to the previous state.

Run the following command to generate your database:

```
$ alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> 6b9cb35ba46e, Empty Init
***
```

This upgrades the database to the `head`, or newest revision.

Now that we've laid down a base for our migrations, we can begin adding data to our SQLAlchemy app. When we make changes to data models, we can use Alembic to automatically generate migrations for us and upgrade the database accordingly.

Autogenerating a Migration

Now let's add our data model to `models.py`. We'll keep working with the `Student` model from previous lessons.

```
# models.py
from datetime import datetime

from sqlalchemy import create_engine, desc
from sqlalchemy import (CheckConstraint, UniqueConstraint,
    Column, DateTime, Integer, String)

from sqlalchemy.ext.declarative import declarative_base

engine = create_engine('sqlite:///migrations_test.db')

Base = declarative_base()

class Student(Base):
    __tablename__ = 'students'
    __table_args__ = (
        UniqueConstraint('email',
            name='unique_email'),
        CheckConstraint('grade BETWEEN 1 AND 12',
            name='grade_between_1_and_12')
    )

    id = Column(Integer(), primary_key=True)
    name = Column(String(), index=True)
```

```
    email = Column(String(55))
    grade = Column(Integer())
    birthday = Column(DateTime())
    enrolled_date = Column(DateTime(), default=datetime.now())

    def __repr__(self):
        return f"Student {self.id}: " \
            + f"{self.name}, " \
            + f"Grade {self.grade}"
```

▶ *Note that we are no longer including the shebang. Why is that?*

With our `Student` model added in, we can now create a migration to add the `students` table to our database. This is a simple migration, so we can take advantage of Alembic's ability to generate the code for us automatically:

```
$ alembic revision --autogenerate -m "Added Student model"
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.autogenerate.compare] Detected added table 'students'
INFO  [alembic.autogenerate.compare] Detected added index 'ix_students_name' on '['name']'
  Generating /python-p3-migrations-and-alembic/app/migrations/versions/361dae855898_added_model.py ...  done
```

During autogeneration, Alembic inspects the metadata of `Base` in `models.py`, comparing it to the current state of the database. In the command above, Alembic detected that we added a model for a `students` table. It also detected that we added an index on `name` to speed up searches by name.

Let's take a look at the new migration file:

```
"""Added Student model

Revision ID: 361dae855898
Revises: 6b9cb35ba46e
```

```
Create Date: 2022-08-04 14:21:32.441071

"""
from alembic import op
import sqlalchemy as sa


# revision identifiers, used by Alembic.
revision = '361dae855898'
down_revision = '6b9cb35ba46e'
branch_labels = None
depends_on = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('students',
    sa.Column('id', sa.Integer(), nullable=False),
    sa.Column('name', sa.String(), nullable=True),
    sa.Column('email', sa.String(length=55), nullable=True),
    sa.Column('grade', sa.Integer(), nullable=True),
    sa.Column('birthday', sa.DateTime(), nullable=True),
    sa.Column('enrolled_date', sa.DateTime(), nullable=True),
    sa.CheckConstraint('grade BETWEEN 1 AND 12', name='grade_between_1_and_12'),
    sa.PrimaryKeyConstraint('id'),
    sa.UniqueConstraint('email', name='unique_email')
    )
    op.create_index(op.f('ix_students_name'), 'students', ['name'], unique=False)
    # ### end Alembic commands ###


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
```

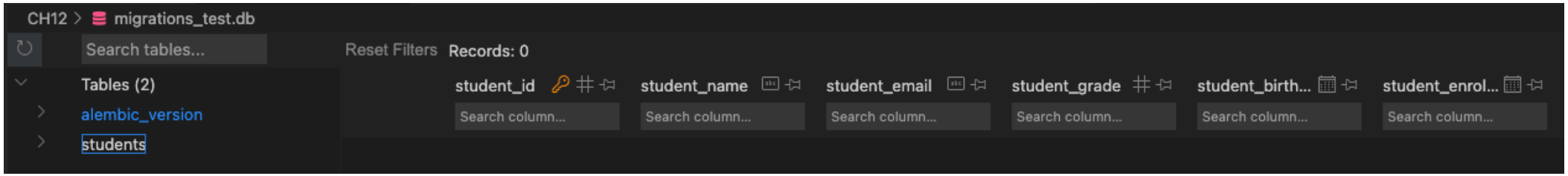
```
    op.drop_index(op.f('ix_students_name'), table_name='students')
    op.drop_table('students')
    # ### end Alembic commands ###
```

There are a few important things to note here. First, a `down_revision` has been added. This points to the ID for the previous migration file. We can also see that the `upgrade()` and `downgrade()` methods have been filled out. The syntax is very similar to that of a SQLAlchemy ORM model- columns, data types, constraints and so on are defined by classes imported via the `sqlalchemy` module. Alembic then executes these instructions using its own `op` class.

Now, run the autogenerated migration to create the `students` table:

```
$ alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade 6b9cb35ba46e -> 361dae855898, Added Student model
```

Open up `migrations_test.db` and you should see two tables: `alembic_version`, which stores the migration ID for the current state of the database, and `students`, which contains all of the columns, keys, and constraints that we included in our model!



Conclusion

Managing migrations is an important skill for a full-stack developer to keep in their toolbox. Alembic is a powerful tool for carrying out migrations and can handle most tasks automatically. That being said, Alembic can't do *everything* on its own. In the next lesson, we will explore how to manually configure migrations and downgrade to revert to an earlier state.

Solution code

```
# models.py

from datetime import datetime

from sqlalchemy import create_engine, desc
from sqlalchemy import (CheckConstraint, UniqueConstraint,
    Column, DateTime, Integer, String)

from sqlalchemy.ext.declarative import declarative_base

engine = create_engine('sqlite:///migrations_test.db')

Base = declarative_base()

class Student(Base):
    __tablename__ = 'students'
    __table_args__ = (
        UniqueConstraint('email',
            name='unique_email'),
        CheckConstraint('grade BETWEEN 1 AND 12',
            name='grade_between_1_and_12')
    )

    id = Column(Integer(), primary_key=True)
    name = Column(String(), index=True)
    email = Column(String(55))
    grade = Column(Integer())
    birthday = Column(DateTime())
    enrolled_date = Column(DateTime(), default=datetime.now())
```

```
    def __repr__(self):
        return f"Student {self.id}: " \
            + f"{self.name}, " \
            + f"Grade {self.grade}"


# alembic.ini
# line 58
sqlalchemy.url = sqlite:///migrations_test.db



# env.py

# migrations/env.py
# lines 21-22
from models import Base
target_metadata = Base.metadata
```

Resources

- **SQLAlchemy ORM Documentation (https://docs.sqlalchemy.org/en/14/orm/)**
- **SQLAlchemy ORM Column Elements and Expressions (https://docs.sqlalchemy.org/en/14/core/sqlelement.html)**
- **Tutorial - Alembic (https://alembic.sqlalchemy.org/en/latest/tutorial.html)**